

Pós-Graduação em Sistemas e Agentes Inteligentes

Construção de APIs para Inteligência Artificial

Rogério Rodrigues Carvalho

INF
INSTITUTO DE
INFORMÁTICA



UFG
UNIVERSIDADE
FEDERAL DE GOIÁS



Sumário

1. Plano de ensino
2. Apresentação
3. Fundamentos e conceitos de APIs
4. Arquitetura de APIs
5. Exemplos de APIs
6. Primeiros passos para desenvolvimento de APIs
7. Atividade prática



Plano de ensino



Expectativas



Apresentação

Apresentação

- Graduação em Sistemas de Informação - IFG
 - TCC: [Desenvolvimento de um sistema de predição educacional utilizando Aprendizagem de Máquina](#)
- Especialização em Design de Sistemas e Soluções de Business Intelligence - INF/UFG
 - TCC: [Sistema de auto machine learning aplicado à predição de desempenho de ações para o mercado de valores BM&F Bovespa](#)
- Mestrado em Ciência da Computação - INF/UFG
 - Dissertação: [Classificação de documentos da administração pública utilizando inteligência artificial](#)
- Analista de TI na UFG desde 2017
 - Líder técnico da plataforma de análise de dados da UFG: Analisa UFG (analisa.ufg.br)

Apresentação

- Atuo em projetos do CEIA/INF
 - Sissa: sisa.ufg.br
 - LicIA
- Atuação em projetos de outros países
- Palestrante convidado na Campus Party Goiás 2019
 - Como agregar valor na vida das pessoas com Machine Learning e Data Science: Cases de sucesso com a utilização de Python
- Mentor convidado no Hackaton da NASA realizado em Goiânia
- Mentor na Maratona de Inovação da UFMT
- LinkedIn: <https://www.linkedin.com/in/rogerionrc/>
- Github: <https://github.com/rogerior>



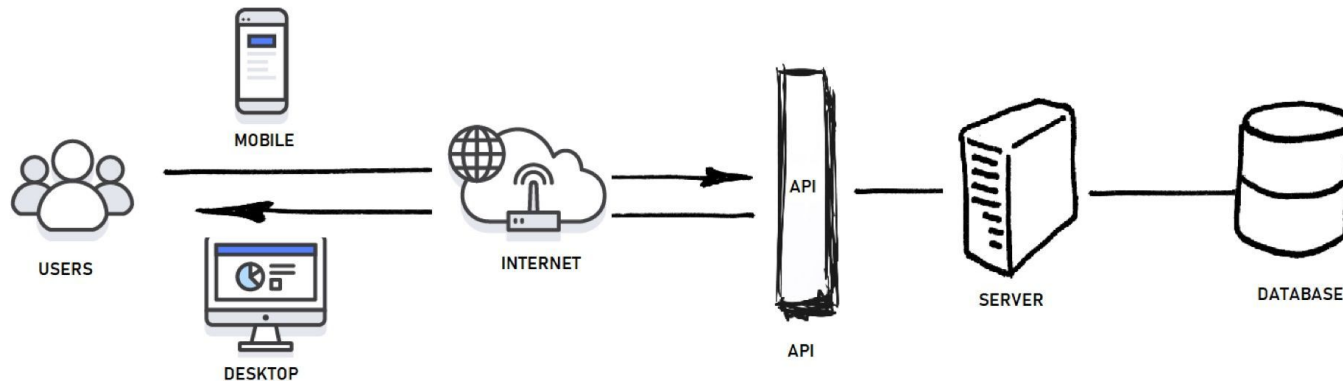
Fundamentos e Conceitos de APIs

O que é uma API?

- Palpites?
- Alguém já utilizou?
- Analogias?

História e Evolução das APIs

- API: Application Programming Interface
- Permitir uma comunicação padronizada entre diferentes sistemas
- <https://traefik.io/blog/the-history-and-evolution-of-apis/>



História e Evolução das APIs

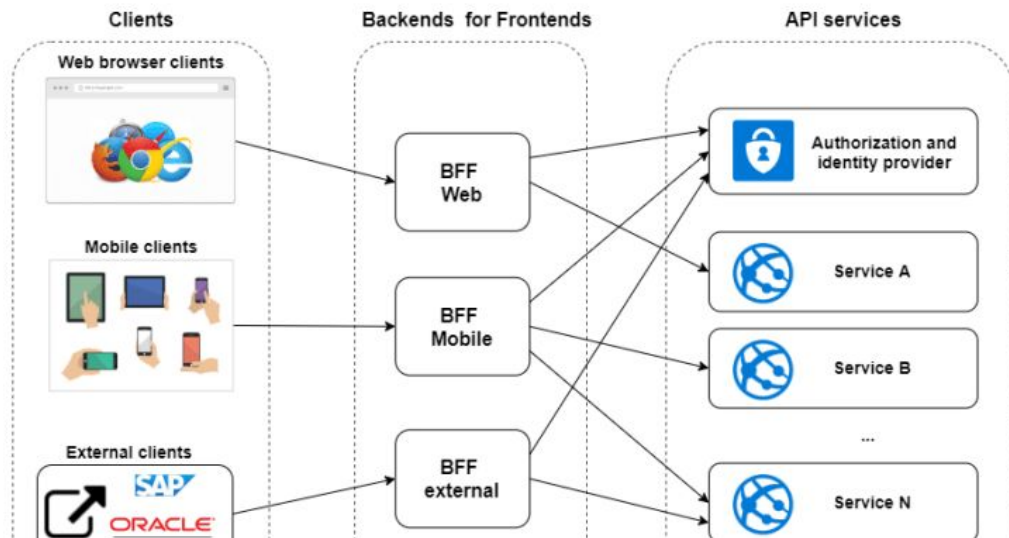
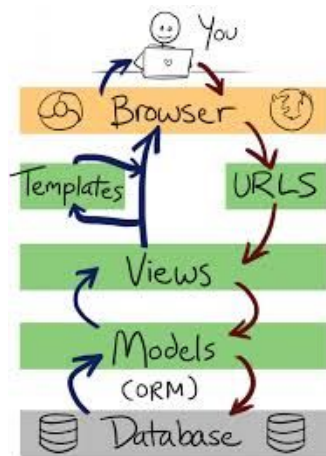


História e Evolução das APIs

- Papel fundamental das APIs na transformação digital e criação de ecossistemas
- Conexão entre aplicações internas (microserviços) ou entre empresas (B2B)
- APIs se tornaram produtos

História e Evolução das APIs

- MVC / MTV
- Back-end e Front-end separados



Tipos de APIs e protocolos comuns

- SOAP (Simple Object Access Protocol)
 - Mais antigo, estrutura baseada em XML
 - Ainda utilizado em sistemas legados e ambientes corporativos

```
1. <?xml version="1.0"?>
2. <soap:Envelope
3.   xmlns:soap="https://www.example.com/soap-envelope/"
4.   soap:encodingStyle="http://www.example.com/soap-encoding">
5.   <soap:Header>
6.     <m:Trans xmlns:m="https://www.example.com/transaction/"
7.       soap:mustUnderstand="a">bcd
8.   </soap:Header>
9.   <soap:Body>
10.    <m:GetEmployee xmlns:m="https://www.example.com/prices">
11.      <m:EName>Joe</m:EName>
12.    </m:GetEmployee>
13.  </soap:Body>
14. </soap:Envelope>
```

Tipos de APIs e protocolos comuns

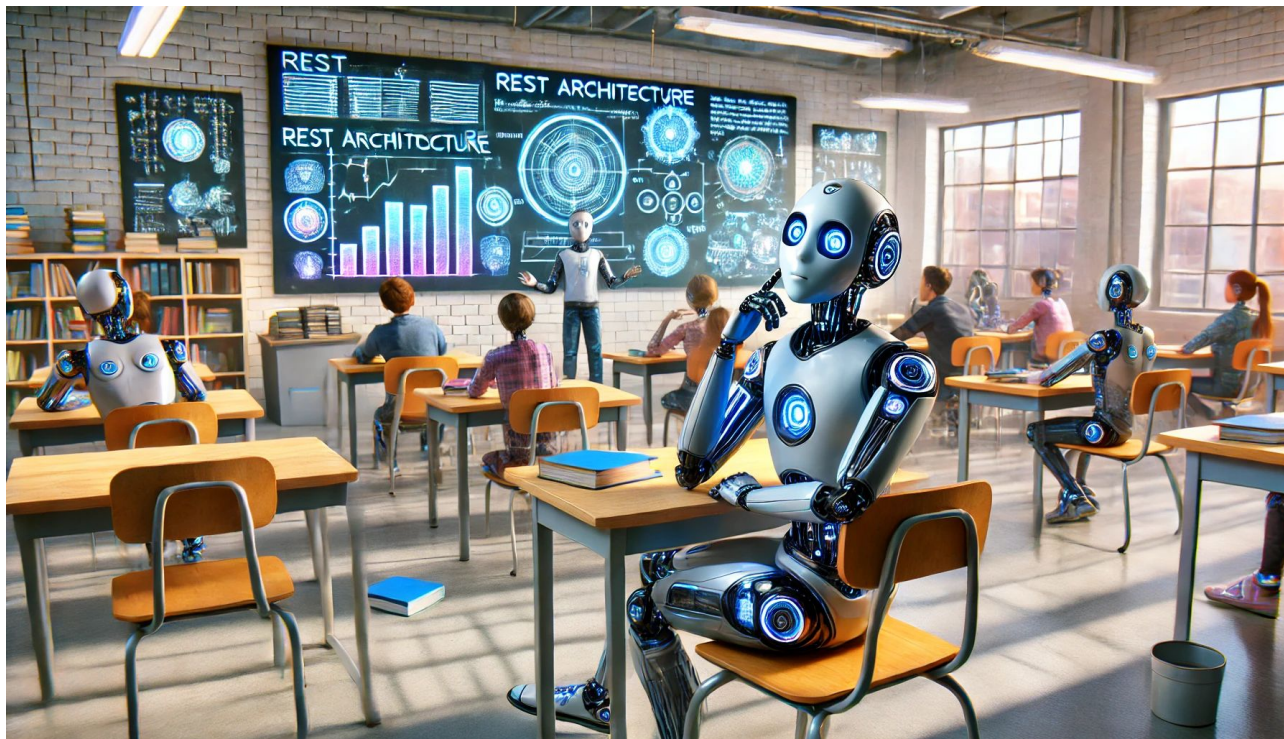
- REST (Representational State Transfer)
 - Baseada em recursos e métodos HTTP (GET, POST, PUT, DELETE).
 - Dados em JSON ou XML

```
{
  "id": 1234,
  "customer": {
    "firstName": "fname1",
    "lastName": "lname1"
  },
  "orderLine": [
    {
      "lineNumber": 1,
      "item": "laptop"
    },
    {
      "lineNumber": 2,
      "item": "iphone"
    }
  ]
}
```

Tipos de APIs e protocolos comuns

- Tendências:
- GraphQL (Facebook)
 - Permite especificar exatamente quais campos e dados deseja receber, evitando requisições muito grandes ou muito limitadas
- gRPC (Google)
 - Foco em alto desempenho, similar a uma estrada expressa onde os dados trafegam em formato binário mais rápido
 - <https://aws.amazon.com/pt/compare/the-difference-between-grpc-and-rest/>

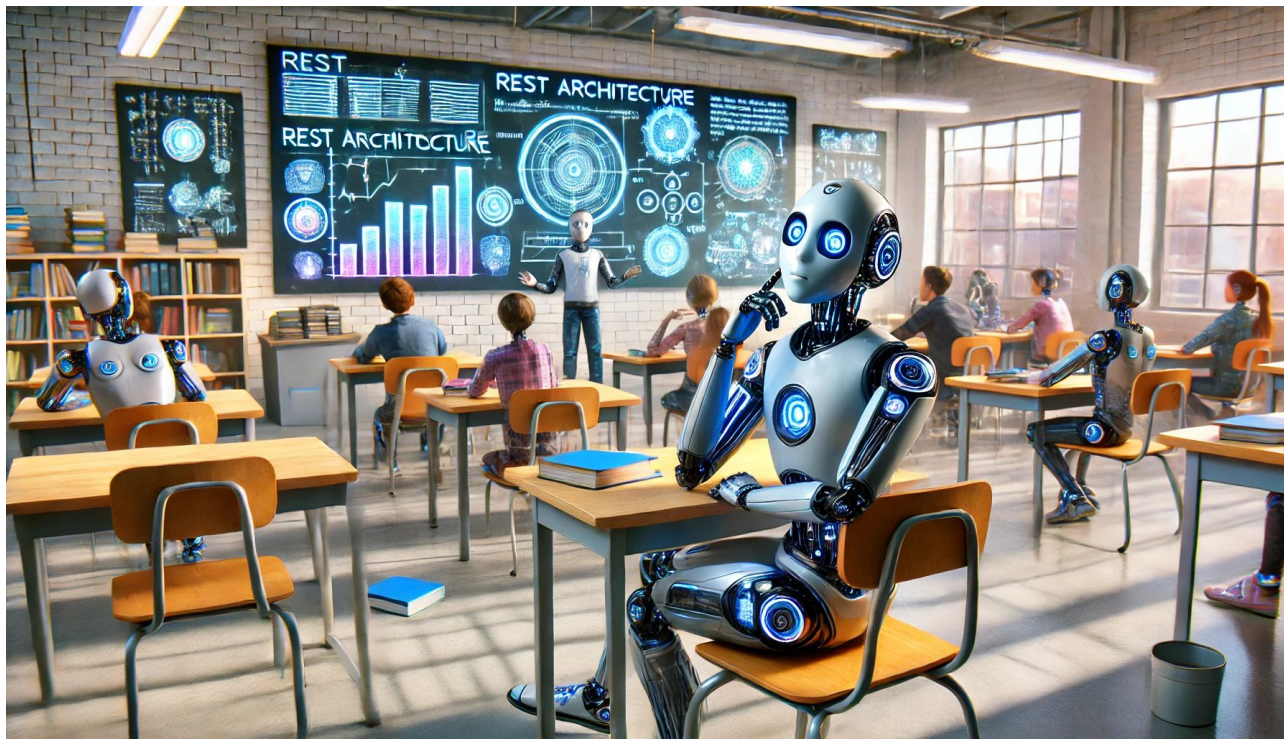
Benefícios em utilizar APIs



Benefícios em utilizar APIs

- Padronização
- Modularidade e reuso de código
- Interoperabilidade
- Separação de responsabilidades (Front-end vs back-end)
- Desacoplamento
- Escalabilidade
- Democratiza acesso a serviços
- Rapidez de desenvolvimento
- Abordagem API-first
 - Google
- Importância para Inteligência Artificial
 - Facilita o consumo de modelos de IA em diferentes sistemas

Desvantagens em utilizar APIs



Desvantagens em utilizar APIs

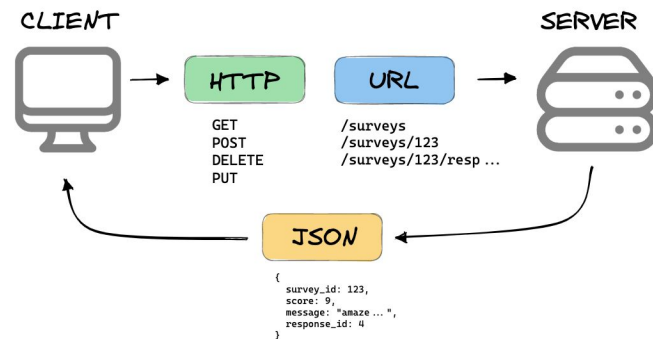
- Complexidade de Integração
 - Utilizar múltiplos serviços externos aumenta a complexidade
- Sobrecarga de Comunicação
 - Chamadas HTTP sucessivas podem tornar as operações mais lentas quando comparadas a chamadas internas
 - Dependência de rede pode introduzir latência, instabilidade e problemas de disponibilidade
- Manutenção e Versionamento
 - A API precisa ser versionada para não quebrar integrações existentes quando novas funcionalidades ou mudanças são realizadas
 - Garantir compatibilidade retroativa

Desvantagens em utilizar APIs

- Dependência de Terceiros
 - Se a API consumida é de um serviço externo, qualquer alteração, instabilidade ou limitação desse provedor afeta diretamente sua aplicação
 - Possíveis custos ou limitações de uso (rate limiting, quotas de chamadas)
- Segurança e Proteção de Dados
 - Endpoints abertos para clientes externos expõe o sistema a riscos de invasões e ataques de DDoS
 - Exige a implementação contínua de práticas de segurança (autenticação, criptografia, monitoramento)
- Gestão de Credenciais e Políticas de Acesso
 - Erros na configuração de permissões podem expor dados críticos ou permitir alterações indevidas.

Terminologia comum em APIs

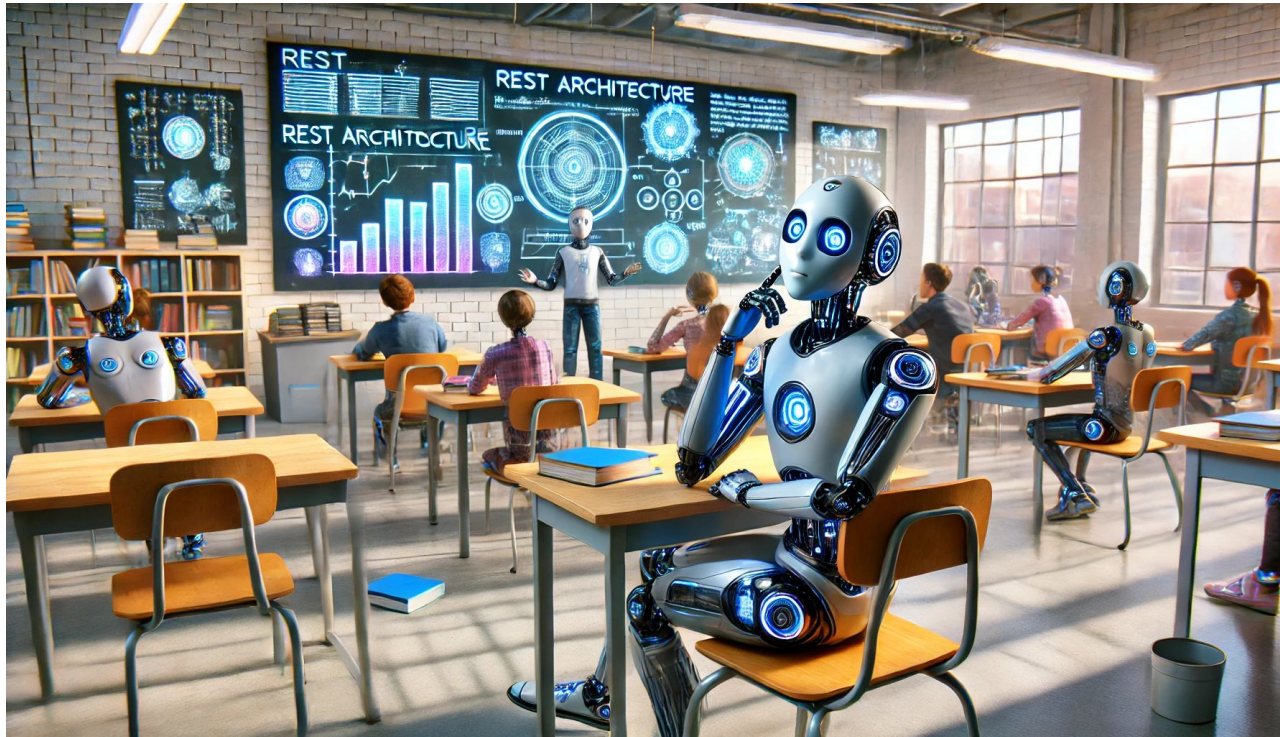
- Endpoint: URL que recebe as requisições (ex.: /api/v1/users)
- Recurso (Resource): o que a API expõe (usuários, produtos, pedidos etc.)
- Métodos HTTP
 - GET: ler/obter recursos
 - POST: criar recursos
 - PUT: atualizar recursos
 - PATCH: atualização parcial
 - DELETE: remover recursos
- Status Codes
 - 2xx: sucesso (ex.: 200 OK, 201 Created).
 - 4xx: erros do cliente (ex.: 400 Bad Request, 404 Not Found).
 - 5xx: erros do servidor (ex.: 500 Internal Server Error).
- Payload / Corpo da requisição: JSON, XML, etc





Arquitetura de APIs

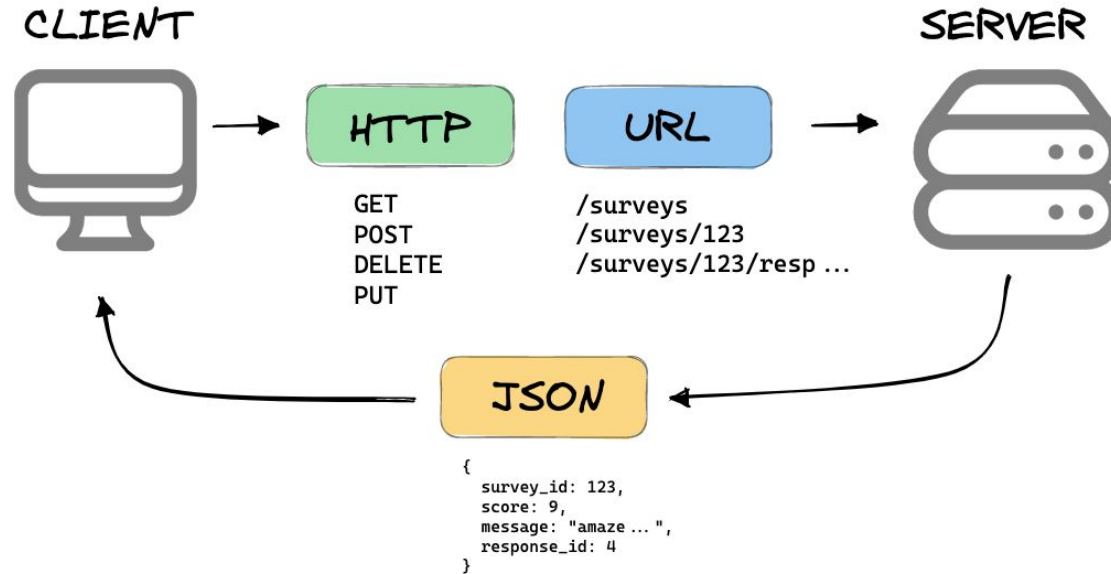
Pilares da arquitetura de APIs



Pilares da arquitetura de APIs

- Escalabilidade
 - Projetar para suportar crescimento de usuários e utilização
- Confiabilidade
 - Lidar com falhas, tolerância a erros, redundância
- Segurança
 - Autenticação, autorização, criptografia, boas práticas
- Manutenibilidade
 - Código limpo, padronização, versionamento, reutilização, documentação

Arquitetura RESTful



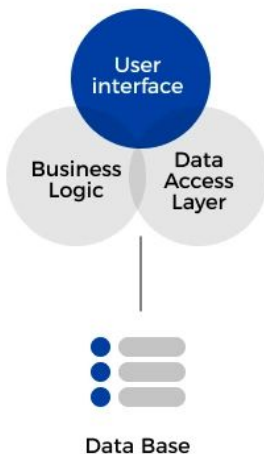
Camadas de uma API

- Camada de Entrada (Controllers ou Routers)
 - Recebe as requisições
- Camada de Negócio (Services)
 - Aplicação da lógica de negócio
- Camada de Acesso a Dados (Models/ORM)
 - Comunicação com o banco de dados

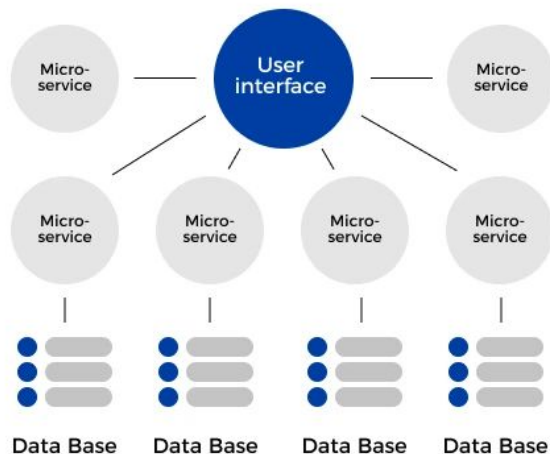
Padrões de projeto em APIs

- Monólitos
- Microsserviços

MONOLITHIC ARCHITECTURE



MICROSERVICE ARCHITECTURE





Exemplos de APIs

Exemplos de APIs

- Sissa
 - <https://api.sissa.ufg.br/>
- OpenAI
 - <https://developers.openai.com/api/docs>
- Google
- Twitter / X
- IBGE
 - <https://servicodados.ibge.gov.br/api/docs>
- Free API
- MGI - PGD
 - <https://github.com/gestaogovbr/api-pgd>



Desenvolvimento de APIs

Hello world

- FastAPI
 - <https://github.com/fastapi/fastapi>

Hello world

- FastAPI
 - Inicialização utilizando o gerenciador de pacotes UV
 - `pip install uv`
 - `uv init`
 - Instalar o fastapi
 - `uv add "fastapi[standard]"`
 - Ativar o ambiente virtual (caso não tenha sido ativado automaticamente)
 - No windows: `.venv\Scripts\activate`
 - No linux: `source .venv/bin/activate`
 - Copie o código de exemplo do github do fastapi: [link](#)
 - Rode o fastapi
 - `fastapi dev`
 - Acesse via navegador
 - <http://127.0.0.1:8000/docs>
 - <http://localhost:8000/docs>



Atividade prática

Utilização de API

- Postman
 - <https://www.postman.com/>
- OpenAI
 - <https://platform.openai.com/api-keys>
 - <https://developers.openai.com/api/docs>
- Groq
 - <https://console.groq.com/keys>
 - <https://console.groq.com/docs/overview>
- IBGE
 - <https://servicodados.ibge.gov.br/api/docs>

Obrigado

Dúvidas?

rogerior@ufg.br
62 986058772

INF
INSTITUTO DE
INFORMÁTICA



UFG
UNIVERSIDADE
FEDERAL DE GOIÁS